TRANSURBAN — guia de apresentação no auditório, sem slides

Apresentação de 20/10/2026. Usar o Swagger e, brevemente, o VS Code. Duração sugerida: 8 a 10 minutos, incluindo os testes, além de perguntas. Falem com calma; os cliques também fazem parte do tempo. Os tempos dependem do ensaio. A divisão abaixo distribui as falas; não atribui autoria individual do código.

1. Decisão para a demonstração

A opção recomendada é levar o notebook Windows do líder, com Python 3.12, dependências instaladas, API ensaiada e a fonte de alimentação. Conectar o notebook ao cabo do projetor, se a escola permitir. Assim não é necessário instalar a API no computador do auditório. Uma saída HDMI pode conectar diretamente; USB-C exige suporte a vídeo e adaptador compatível. Peça ao responsável para conferir o cabo e a entrada disponíveis.

Na apresentação: SQLite local para os cadastros DS, CSV público histórico de Curitiba para IA e Swagger servido pelo próprio projeto. A base sintética anterior fica separada no grupo “IA — Base sintética anterior (testes)” e não é o conjunto apresentado nesta revisão. O inicializador iniciar_apresentacao.py define a conexão local no processo; não precisa de .env. O banco local é um banco de dados SQL em arquivo, com PK e FK; não é um banco Supabase. O arquivo .env é configuração privada para outras formas de iniciar, não um requisito para esta demonstração.

A estrutura do Supabase foi conferida por capturas enviadas pelo usuário, incluindo PK/FK, geração automática de IDs e RLS. A conexão Python com o pooler foi interrompida antes da autenticação. Não há comprovação de POST/GET remoto. Não tentar resolver senhas, rede ou TLS no palco.

2. Preparação no Windows, antes de sair de casa

Baixe a versão atual da branch entrega/escola-apresentacao-2026-10-20 e extraia o ZIP. Não use uma cópia antiga sem swagger_static. Abra o terminal do VS Code na pasta backend_escola. Não copie o ambiente virtual Linux para Windows.

Execute um comando de cada vez e confira o resultado antes de continuar:

```
py -3.12 iniciar_apresentacao.py --preparar
py -3.12 iniciar_apresentacao.py --testar
py -3.12 iniciar_apresentacao.py --evidencias
py -3.12 iniciar_apresentacao.py
```

Atalhos opcionais no Windows: na pasta backend_escola, PREPARAR-WINDOWS.cmd executa instalação, testes e evidências e para se houver falha. APRESENTAR-WINDOWS.cmd inicia a API já preparada. Os atalhos foram fornecidos para conveniência; a execução Windows precisa ser ensaiada nesse notebook.

A preparação requer internet e Python 3.12. Os testes devem passar sem falhas; o número de referência nesta revisão é 29 testes. O aviso de depreciação de Starlette/httpx, se aparecer, não é falha. O gerador deve indicar 22 requisições verificadas. A contagem deve ser confirmada no notebook: a validação na nuvem não substitui a execução nele.

Aguarde Application startup complete, deixe o terminal aberto e digite no navegador DO MESMO NOTEBOOK:

- http://127.0.0.1:8000/health: deve mostrar status ok e banco sqlite_local.

- http://127.0.0.1:8000/docs: Swagger para os testes.

- /painel/: painel opcional; não é necessário na apresentação.

O inicializador ativa o modo de apresentação: a documentação /docs mostra o roteiro, DS e os dados públicos; não lista as rotas sintéticas anteriores. Elas continuam disponíveis para os testes, sem serem apagadas. GET /apresentacao mostra a sequência e os resultados esperados.

A partir desta revisão, /docs carrega JS e CSS locais de swagger_static, em vez de CDN. Depois da instalação, faça um ensaio com o Wi-Fi desligado: abra uma nova aba, carregue /docs e execute /health e /ia/dados-publicos/resumo. O banco e o CSV devem funcionar sem internet. Ligue a rede novamente se precisar baixar ou enviar arquivos; não execute instalação durante a apresentação.

Pare com Ctrl+C, inicie novamente com o último comando e faça GET /linhas. Confira que os cadastros do ensaio permaneceram no arquivo apresentacao_local.db. Não apague esse arquivo sem necessidade. IDs podem mudar ao repetir cadastros; use sempre o ID realmente retornado.

Leve também o ZIP atualizado num pendrive, o roteiro e os JSONs das evidências. Como a instalação de dependências exige internet, um pendrive com o código sozinho não garante que um computador novo esteja preparado.

3. Projetor e organização do palco

1. Chegue antes do público. Peça ao responsável para conectar a saída do notebook à entrada correta do projetor. Não altere instalações ou cabeamento fixo sem orientação.

2. No Windows, pressione Win+P e escolha Duplicar. O telão mostrará a mesma tela do notebook. Se não aparecer, confira com o responsável a entrada selecionada e o adaptador; o fato de o cabo entrar não garante compatibilidade.

3. Conecte o carregador, desative a suspensão durante a apresentação e silencie notificações.

4. Inicie a API e abra /docs no navegador do notebook. Não use o celular para abrir 127.0.0.1: ele aponta para o próprio aparelho.

5. Ajuste o zoom do Chrome, por exemplo para 125% ou 150%, e confira a leitura no fundo da sala. Use F11 se ajudar. Evite mostrar páginas longas de JSON sem indicar o campo explicado.

6. Deixe database.py, models.py, schemas.py e main.py abertos no VS Code. Nunca projete .env, senhas ou autenticações.

7. José Henrique pode operar o computador durante todas as falas. Bruno e Kauan apenas falam e podem ler suas frases. Cada colega espera o resultado da tela antes de anunciar o que aconteceu.

8. Deixe os exemplos deste roteiro em um arquivo de texto local. Copie somente o JSON necessário, com calma, sem editar código no palco.

Se tiverem de usar o computador Linux do auditório, preparem-no antes da entrada do público, com Python 3.12. Na pasta backend_escola:

```
python3 iniciar_apresentacao.py --preparar
python3 iniciar_apresentacao.py --testar
python3 iniciar_apresentacao.py --evidencias
python3 iniciar_apresentacao.py
```

Confira antes que python3 --version indica 3.12; não execute esses comandos às cegas em uma versão diferente. Não transfira .venv-apresentacao do Windows para Linux. Use o mesmo navegador e a mesma URL no computador que estiver executando a API.

4. Ordem e apoio de cada integrante

Ordem	Integrante	Responsabilidade	Apoio na tela
Abertura	José Henrique, líder	Resumo do projeto e limites da demonstração	Swagger aberto, sem clicar
1	Bruno	Objetivo, fala curta	Não opera o computador
2	Kauan	GET, fala curta	Não opera o computador
3	José Henrique	Arquivos, PK/FK, POST e GET	VS Code, depois Swagger
4	Gustavo	Validações e erros esperados	404, consulta e 422
5	Jullyany	Fonte pública e qualidade	GET /ia/dados-publicos/fonte e /qualidade
6	Camila	Contagem, média, mediana e comparação	Resumo público, linha e serviço
7	Ana Carolina	Evidências, GitHub e conclusão	Roteiro ou resultados já exibidos

Como usar as falas

O texto entre aspas é para falar. As instruções fora das aspas são só para quem opera o computador. José Henrique pode executar todos os cliques; cada colega explica a sua parte. Apontem para o campo explicado, esperem o resultado e só então continuem. Não leiam o JSON inteiro nem recitem endereços de rotas para o público. Se esquecerem uma frase, expliquem a ideia com suas próprias palavras.

Abertura — José Henrique, líder (cerca de 1 minuto)

“Bom dia, pessoal! Eu sou o José Henrique e esta é a nossa equipe: Bruno, Kauan, Gustavo, Jullyany, Camila e Ana Carolina. Hoje vamos apresentar o TRANSURBAN, um projeto pensado para a mobilidade entre Maringá e Sarandi.

Vamos mostrar a parte que trabalha por trás de uma interface: a API. Ela recebe um pedido, confere as informações e devolve uma resposta. Por exemplo: cadastrar uma cidade ou consultar uma linha de ônibus.

Nossa apresentação tem duas partes. Primeiro, vamos cadastrar e consultar informações e testar o que acontece quando alguém envia um dado inválido. Depois, vamos mostrar como o Python transforma um arquivo de dados em números que podemos analisar.

Nessa análise, usamos uma base pública histórica de Curitiba como estudo externo. A fonte não é oficial da URBS e não contém atrasos medidos. Vamos deixar claro o que os dados mostram e o que não permitem concluir.

O Bruno vai começar explicando o objetivo dos cadastros.”

Parte 1 — Bruno (15 a 20 segundos)

“Nosso sistema organiza cidades e linhas de ônibus. Primeiro cadastramos a cidade. Depois, cadastramos uma linha ligada a ela. Assim conseguimos consultar a linha e saber a qual cidade ela está vinculada. Agora o Kauan vai explicar a consulta.”

Bruno só lê ou fala essas frases. Não precisa operar o computador ou explicar bibliotecas.

Parte 2 — Kauan (15 a 20 segundos)

“Consultar é pedir para ver uma informação que já está salva. Na API, usamos GET para isso. Ele mostra os dados sem fazer um novo cadastro. O José Henrique vai mostrar como cadastramos e consultamos na prática.”

Kauan só lê ou fala essas frases. Não precisa executar comandos.

Parte 3 — José Henrique: cadastro e relacionamento (cerca de 2 minutos e meio)

Antes de falar: preparar a tela

Confira GET /apresentacao e GET /health antes de chamar a equipe. Na fala, basta apontar que /health retornou status ok e banco sqlite_local. Se o professor pedir, abra as respostas. Os testes da demonstração usam banco local; não há conexão Supabase comprovada.

Passo 1: explicar a organização, mostrando o VS Code brevemente

“Organizamos o código em partes: conexão com o banco, tabelas, validação dos dados e rotas. Isso facilita entender a função de cada arquivo e encontrar onde fazer uma alteração.

O projeto usa Python. O FastAPI disponibiliza as rotas, o SQLAlchemy trabalha com o banco e o Pydantic valida os dados recebidos. A análise do arquivo público usa Pandas.”

Quem opera aponta, sem ler o código inteiro: database.py = conexão; models.py = tabelas e relações; schemas.py = validação; main.py = rotas. A análise pública atual está em dados_publicos.py; importar_dados_publicos.py produz o CSV. analise.py/ia_routes.py pertencem à base sintética anterior e não são os arquivos da nova análise pública.

Passo 2: cadastrar a cidade

Volte ao Swagger. Em POST /cidades, clique Try it out, selecione o exemplo de cadastro válido e Execute:

```
{"nome":"Sarandi"}
```

Espere Server response 201. Aponte para nome e id_cidade, sem assumir que o ID é 1.

“Agora cadastramos Sarandi. POST é o método que usamos para enviar esse cadastro. O código 201 confirma que o registro foi criado.

Este número é a identificação da cidade no banco. Ele é a chave primária: permite identificar cada registro sem confundir um com outro.”

Passo 3: cadastrar a linha usando o ID retornado

Em POST /linhas, selecione o exemplo válido, substitua id_cidade pelo número realmente devolvido e Execute. O 1 abaixo é só ilustrativo:

```
{"codigo":"DEMO-20","nome":"Linha da apresentação","id_cidade":1}
```

Espere 201. Aponte para id_cidade e cidade.

“Vamos cadastrar uma linha de demonstração usando a identificação da cidade. Esse nome e esse código foram escolhidos apenas para o teste.

O campo id_cidade liga a linha ao registro da cidade. Essa ligação é a chave estrangeira. Neste recorte do sistema, uma cidade pode ter várias linhas, e cada linha precisa apontar para uma cidade cadastrada.”

Passo 4: consultar o que foi salvo

Execute GET /linhas. Esperado: 200, com o cadastro e cidade Sarandi. GET /cidades também fica disponível se quiserem conferir a cidade.

“Agora consultamos o que acabamos de salvar. O código 200 indica que a consulta foi atendida. A resposta traz o nome da cidade junto da linha, para facilitar a leitura.

Esse formato de campos e valores se chama JSON. Uma interface pode usar essa resposta para mostrar as informações ao usuário. O Gustavo vai testar o que acontece quando os dados estão errados.”

Repetir cadastro pode criar outro ID: use sempre o retornado nessa execução. Não apagar o banco do ensaio nem afirmar que SQLite rejeita nomes repetidos.

Parte 4 — Gustavo: validações (cerca de 1 minuto)

Passo 1: provocar uma referência inválida

Confira antes por GET /cidades que não existe o ID 999999. Em POST /linhas, selecione o exemplo de cidade inexistente:

```
{"codigo":"TESTE-ERRO","nome":"Tentativa inválida","id_cidade":999999}
```

Fale antes de clicar Execute:

“Até aqui usamos dados válidos. Agora vamos enviar uma cidade que não está cadastrada, de propósito, para testar a proteção da API.”

Depois de aparecer 404 e Cidade não encontrada:

“A API identificou que a cidade não existe e recusou o cadastro. Esse 404 é o resultado esperado do nosso teste.”

Execute GET /linhas e confirme que TESTE-ERRO não aparece.

“A consulta mostra que a tentativa inválida não foi salva.”

Passo 2: testar nome vazio

Em POST /cidades, selecione o exemplo de nome somente com espaços e Execute:

```
{"nome":"   "}
```

Depois de aparecer 422:

“Agora enviamos um nome com apenas espaços. A validação rejeitou esse valor e retornou 422. Isso evita guardar uma cidade sem nome. A Jullyany vai explicar os dados usados na análise.”

Parte 5 — Jullyany: de onde vêm os dados (cerca de 1 minuto)

Passo 1: executar GET /ia/dados-publicos/fonte

Aponte para publicador_gtfs, download_url e periodo_declarado_feed. Não precisa decorar todos os nomes de instituições ou ler o hash em voz alta.

“Para a análise, usamos um arquivo público de programação do transporte de Curitiba. Ele foi publicado pelo projeto comunitário URBS-GTFS e está catalogado na Mobility Database. A origem e o link para conferência aparecem aqui.

O período informado vai de 28 de setembro a 5 de outubro de 2026. Portanto, é uma base histórica de Curitiba. Ela não é um feed oficial da URBS e não representa a operação de Maringá e Sarandi.”

Passo 2: executar GET /ia/dados-publicos/qualidade

Aponte para registros_lidos, ids_duplicados, ausentes_por_coluna e apta_para_calculos_do_snapshot. Só anuncie as verificações que aparecerem na resposta.

“O arquivo foi organizado como CSV, uma tabela de dados. Cada linha representa um registro de programação publicado; não uma viagem que comprovadamente aconteceu.

Antes de calcular, conferimos campos vazios, identificadores repetidos e se os arquivos continuam iguais aos que foram importados. Se essas verificações encontrarem um problema, a API bloqueia os indicadores para revisão.

Não criamos tempos de atraso para preencher o que a fonte não informa. A Camila vai mostrar o que podemos calcular com esses registros.”

Se perguntarem a origem primária, use a resposta de apoio no final: o publicador declara usar URBS/C3SL-UFPR; nossa conferência direta foi do derivado publicado, do código e do catálogo. Não afirmar coleta da equipe ou certificação da URBS.

Parte 6 — Camila: explicar os números (cerca de 1 minuto e meio)

Passo 1: executar GET /ia/dados-publicos/resumo

Aponte para um campo de cada vez. Não leia a resposta inteira.

“A primeira pergunta é: o que existe nesse arquivo? Encontramos 71.086 registros de programação, distribuídos por 309 códigos de linha.

Contamos quantos registros pertencem a cada linha. Dividindo o total por 309, temos uma média de aproximadamente 230 registros por linha.

A mediana é 162. Para encontrá-la, ordenamos as 309 contagens e olhamos o valor do meio. Ela ajuda a entender a distribuição junto com a média.

Esses valores são quantidades de registros do arquivo. Não são minutos de atraso nem uma quantidade diária de ônibus.”

O JSON mostra a média com mais precisão: 230,0518. Dizer aproximadamente 230 evita sobrecarregar a fala; se perguntarem o cálculo, mostre 71.086 / 309. Não calcular mediana dos códigos das linhas: a mediana é das contagens por código.

Passo 2: mostrar cinco grupos por linha

Em GET /ia/dados-publicos/agrupamentos/{dimensao}, clique Try it out, selecione linha, preencha limite 5 e Execute.

“Agora separamos os registros por linha para comparar as quantidades. Para facilitar a leitura no telão, mostramos cinco grupos. O total completo continua identificado: são 309 grupos.”

Aponte total_grupos e grupos_exibidos. A soma dos cinco grupos está separada da soma de todos. Não chamar a linha com mais registros de mais movimentada: não há passageiros ou demanda observada nessa análise.

Passo 3: agrupar por serviço

Na mesma operação, selecione servico e Execute. Com limite vazio ou 5, aparecem as três categorias.

“Também podemos separar os registros nas categorias dias úteis, sábado e domingo. Isso mostra como o arquivo está organizado por tipo de serviço.”

Volte ao GET /ia/dados-publicos/resumo e aponte atraso_medio_min. Esse campo não está no agrupamento.

“O campo de atraso médio aparece como null, que significa informação indisponível, não atraso zero. A fonte não tem a medição necessária para esse cálculo. Agora a Ana Carolina vai concluir.”

No resumo estão os campos null; no agrupamento aparecem as três categorias. Se necessário, volte ao resumo para apontar atraso_medio_min. Não espere esse campo no JSON do agrupamento. As categorias somam 71.086: dias úteis 33.425, sábado 20.754, domingo 16.907. Não é total de viagens realizadas em um dia ou semana. Não é necessário recitar esses três números: estão disponíveis para conferência na tela e nas perguntas.

Parte 7 — Ana Carolina: conclusão (cerca de 40 segundos)

“Hoje mostramos três resultados: cadastrar e consultar informações, impedir cadastros inválidos e transformar um arquivo público em indicadores que podem ser conferidos.

A preparação inclui testes automatizados e respostas registradas como evidência. O código e o roteiro estão disponíveis no GitHub.

A demonstração usa banco local e dados públicos históricos de Curitiba. A conexão com o Supabase ainda está pendente. Nosso objetivo nesta etapa é demonstrar o funcionamento do backend e explicar os dados com clareza.

Obrigada pela atenção! Estamos disponíveis para perguntas.”

Informação de apoio, se perguntarem sobre testes: a revisão da API passou em 29 testes e 22 requisições na nuvem Linux. Só afirmar execução no notebook da apresentação depois de conferir os resultados nele. Não confundir contagem de testes com registros de transporte.

5. O que demonstramos de DS e de IA

Área ou disciplina	Evidência concreta nesta entrega
Análise e Métodos de Sistemas	Tema de mobilidade, entidades Cidade/Linha, organização de responsabilidades e requisitos de cadastro/consulta
Banco de Dados I e II	PK, FK, relacionamento um para muitos, gravação, leitura e consulta com nome relacionado
Lógica Computacional	Verificação de existência, rejeição de texto vazio e regras de validação e agrupamento
Programação Backend I	FastAPI, POST/GET, Pydantic, SQLAlchemy, sessões, códigos HTTP e testes
Programação Python I	Módulos, funções, dependências isoladas e processamento em Python
Ciência de Dados	CSV, unidade de análise, qualidade, contagens, soma, média, mediana e agrupamentos com Pandas
Inovação e Tecnologia	Aplicação ao tema de mobilidade e disponibilização de indicadores para uma interface futura

Essa tabela relaciona conteúdos demonstrados; não substitui critérios específicos de correção do professor. O backend escolar tem duas entidades principais; não afirmar que todo o sistema de transporte está implementado.

6. Perguntas prováveis

O que é API? Uma interface com regras de comunicação. Neste projeto recebe requisições HTTP e devolve respostas JSON para acesso às funções do backend.

Swagger é a API? É uma interface de documentação e teste. As rotas continuam acessíveis por outros clientes HTTP.

GET e POST? GET consulta; nossos POSTs cadastram registros.

Por que voltar o nome da cidade se já tem o ID? O ID mantém a referência; o nome torna a resposta compreensível.

Model e schema? Model representa as tabelas e relações; schema valida os dados HTTP e descreve a resposta.

Por que venv? Para isolar bibliotecas e reproduzir as dependências sem misturá-las com outros projetos.

Por que não tem .env? No modo de apresentação, o inicializador define explicitamente SQLite local no processo. Outros modos podem ler a conexão privada no .env. Não é necessário projetá-lo nem distribuí-lo.

Onde ficam os cadastros? Em apresentacao_local.db no notebook que executa a API. Uma consulta após reiniciar confere a persistência no mesmo arquivo. O banco não é compartilhado automaticamente entre notebooks.

O Supabase está conectado? Ainda não foi comprovada a conexão Python. A estrutura foi conferida, mas a demonstração usa SQLite. Não confundir o editor SQL acessível pelo navegador com acesso PostgreSQL pelo Python.

A fonte é oficial e atual? Não. É um feed comunitário público, catalogado como não oficial, com período declarado de 28/09–05/10/2026. A origem primária URBS/C3SL é declarada pelo publicador; não afirmar coleta direta ou validação da URBS.

Por que Curitiba se o projeto é Maringá/Sarandi? Não obtivemos uma base local verificável nesta preparação. Usamos Curitiba como estudo externo para demonstrar o processamento sem fabricar registros locais. Para operar nas cidades do projeto, será necessário obter e validar uma fonte local.

De onde vieram os números de 7,57 e 8 minutos do roteiro antigo? Eram cálculos de uma base sintética de teste. Não são atrasos medidos e não são os indicadores desta apresentação.

Qual a fonte? O endpoint /ia/dados-publicos/fonte mostra o download, o publicador, o catálogo MDB-3225, o período, hashes e limitações. Os arquivos e o script de importação estão no repositório.

Por que usar SQLite? É um banco relacional local adequado à demonstração sem depender de acesso ao banco remoto. Não estamos afirmando que representa a implantação final de produção.

Os 71.086 registros são viagens realizadas? Não. São registros trip_id da programação GTFS convertida. Sem observações de operação, viagens_realizadas é null. Não contamos ônibus únicos ou passageiros.

Esses dados mostram atrasos reais? Não. Não temos tempos observados para isso. A soma e a média apresentadas são de contagens de registros de programação por linha, não de minutos.

O cadastro de uma linha altera os indicadores? Não. O cadastro DS usa SQLite; a análise lê o snapshot público externo de Curitiba. São fontes independentes e não existe sincronização automática.

Tem IA treinada? Nesta avaliação há Ciência de Dados descritiva; não há modelo preditivo treinado.

A análise prova o motivo do atraso? Não. Agrupamentos podem mostrar associações; não comprovam causalidade.

Tudo está na main e publicado na internet? A entrega está na branch indicada. Branch publicada não significa merge na main, API hospedada ou site em produção.

O frontend de passageiro usa esta API? O painel servido em /painel/ usa suas rotas. O protótipo React/Vite da escola e o site V2 são versões diferentes; não afirmar integração sem verificar.

7. Se algo não sair como esperado

- 404 no teste de cidade inexistente e 422 no teste de nome vazio são resultados esperados. Expliquem isso antes e depois da execução.

- Se a API não responder, confira o terminal. Não feche a janela do servidor nem pressione Ctrl+C durante o teste sem intenção de reiniciar.

- Se a porta estiver ocupada, não encerre processos desconhecidos. Inicie com --port 8001 e use 8001 no navegador.

- Se o Swagger estiver em branco, confira se é a cópia atual com swagger_static e se a API está funcionando. Evite tentar consertar bibliotecas no palco.

- Se aparecer outro resultado ou erro inesperado, não invente uma explicação. Diga: “Vamos conferir a execução; temos as respostas dos testes anteriores como apoio.” Use as evidências, identificando de onde vieram.

- Sem internet, não tentar Supabase, GitHub ou instalação. Com dependências e Swagger locais já preparados, a demonstração SQLite não depende desses serviços.

- Se o projetor não exibir, peça ajuda ao responsável pelo equipamento. A API funcionar e o vídeo projetar são verificações diferentes.

- Se o notebook não puder ser conectado, só use o computador do auditório depois de preparar e testar nele. Não existe garantia de funcionamento por apenas copiar um ambiente virtual de outro sistema.

8. Conferência final do líder

Antes de chamar a equipe: notebook carregando; cabo/adaptador conferidos; projeção legível; notificações silenciadas; API iniciada; /health com sqlite_local; Swagger carregado; snapshot público com 71.086 registros e origem aberta em uma aba; ID da cidade anotado; exemplos prontos; falas distribuídas; backup no pendrive. Não mudar o código depois do ensaio sem repetir a verificação afetada.

Esta orientação reduz riscos; não promete ausência de falhas em equipamento que ainda não foi testado. A validação final precisa ocorrer no notebook e na ligação com o projetor que serão usados.